

University of Victoria
Faculty of Engineering
Summer 2026 ENGR 446 — Milestone Report II: Engineering Analysis

**Design and Analysis of a Pipelined RTL Hardware Accelerator
for Real-Time Sobel Edge Detection**

Robin Zerr
V01013510
ENGR 446
Electrical Engineering
Email: zerrobin@gmail.com

July 3, 2026

In partial fulfillment of the requirements of the B.Eng. Degree

Table of Contents

1.0 Proposed Solution.....	1
2.0 Engineering Analysis.....	2
2.1 Real-Time Throughput Requirement.....	2
2.2 Sequential RTL Throughput.....	3
2.3 Pipelined RTL Throughput.....	3
2.4 Latency Analysis.....	4
2.5 Sobel Arithmetic Analysis.....	4
2.6 Resource Usage Analysis.....	5
2.7 Verification Results.....	5
2.8 Sensitivity and Limitations of the Analysis.....	6
3.0 Discussion.....	7
References.....	8
Appendix A: Milestone Report I — Project Background.....	A-1

List of Figures

Figure 1: Block diagram of the pipelined Sobel accelerator.....	1
Figure 2: The 3×3 pixel window and the Sobel kernels.....	2
Figure 3: Pipeline timing.....	2
Figure 4: Verification of the 640×480 run.....	6

List of Tables

Table 1: Expected synthesis results (pre-synthesis estimates).....	5
Table 2: Verification results, RTL versus golden model.....	5

1.0 Proposed Solution

The proposed solution is a pipelined RTL Sobel edge-detection accelerator that processes a continuous stream of 8-bit grayscale pixels and produces one filtered output pixel per clock cycle in steady state. The design is written in VHDL and is organised into the hierarchy shown in Figure 1: a window-forming stage built from two line buffers and a bank of shift registers, followed by a two-stage arithmetic pipeline. Every stage is registered, so a new pixel is accepted on each clock edge once the pipeline has filled [1].

Incoming pixels enter one per clock cycle. The two line buffers each store one full image row, holding the two previous rows so that the three vertically aligned pixels of the current window column are available at the same time. Three 3-tap shift registers then provide the horizontal taps, presenting all nine pixels of the window (p_{00} through p_{22}) to the arithmetic stages in parallel. The window labelling and the two Sobel kernels are shown in Figure 2 [2].

The arithmetic datapath computes the horizontal gradient G_x and the vertical gradient G_y , forms the approximate magnitude $M \approx |G_x| + |G_y|$, and saturates the result to the 8-bit output range. A valid flag travels with the data through every pipeline register, so outputs produced while the buffers are filling, and along the image borders where no complete 3×3 window exists, are marked invalid; the output frame is therefore $(W - 2) \times (H - 2)$ pixels. Figure 3 shows the resulting behaviour: a one-time fill latency followed by one output pixel per clock cycle.

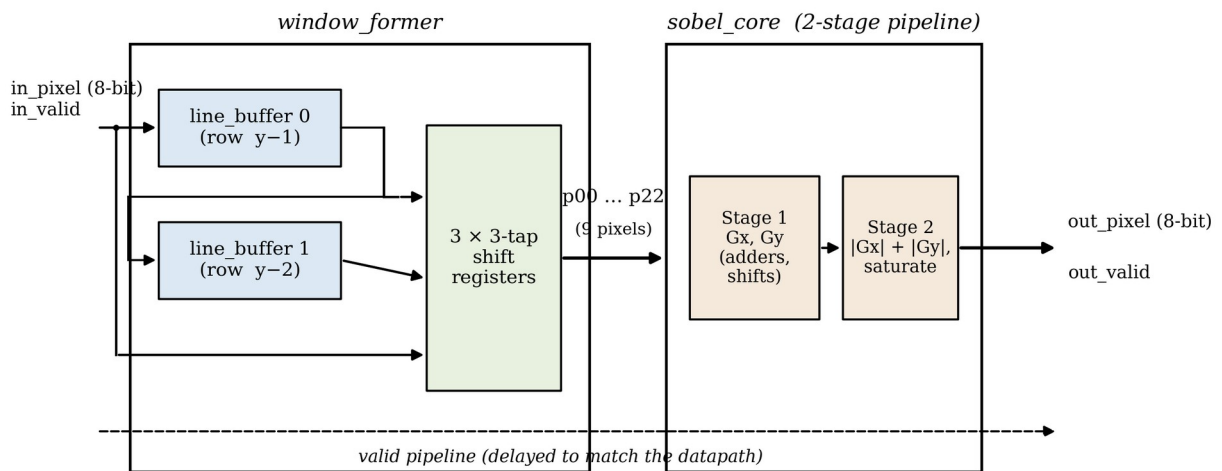


Figure 1: Block diagram of the pipelined Sobel accelerator.

3×3 pixel window			Gx kernel (horizontal)			Gy kernel (vertical)		
p00	p01	p02	-1	0	+1	-1	-2	-1
p10	p11	p12	-2	0	+2	0	0	0
p20	p21	p22	-1	0	+1	+1	+2	+1

Figure 2: The 3×3 pixel window and the horizontal (Gx) and vertical (Gy) Sobel kernels.

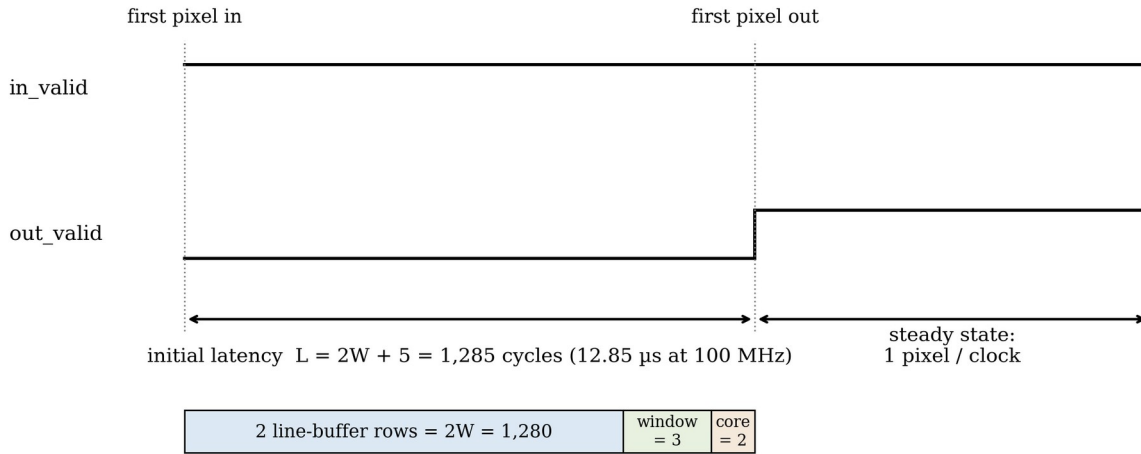


Figure 3: Pipeline timing, showing the fill latency and the one-pixel-per-cycle steady state.

2.0 Engineering Analysis

The analysis proceeds from the application requirement to the measured behaviour of the design. Section 2.1 establishes the pixel rate the accelerator must sustain, Sections 2.2 and 2.3 compare the throughput of the sequential and pipelined architectures, Section 2.4 derives and then measures the pipeline latency, and Section 2.5 examines the Sobel arithmetic and its bit widths. Sections 2.6 through 2.8 cover resource usage, verification against a software golden model, and the sensitivity and limitations of the analysis. All simulation results were obtained with the GHDL simulator (VHDL-2008) using a file-driven testbench and a 100 MHz testbench clock [3].

2.1 Real-Time Throughput Requirement

The first requirement is the sustained pixel rate that the accelerator must support. The number of pixels in one frame is the product of the image width and height, as given in Equation (1).

$$N_{px} = W \times H \quad (1)$$

In Equation (1), N_{px} is the number of pixels in one frame, W is the image width in pixels, and H is the image height in pixels. The required sustained pixel rate is the pixels per frame multiplied by the frame rate, as in Equation (2).

$$R_{px} = W \times H \times F \quad (2)$$

In Equation (2), R_{px} is the required pixel rate in pixels per second and F is the frame rate in frames per second. For the target of 640×480 grayscale video at 30 frames per second, Equation (2) gives a required rate of $640 \times 480 \times 30 = 9.216 \times 10^6$ pixels per second. Any candidate architecture must sustain at least this rate to meet the project aim.

2.2 Sequential RTL Throughput

A sequential architecture shares a small number of arithmetic units across the Sobel computation, so it needs several clock cycles for each output pixel. The exact count depends on how many multiply-accumulate and absolute-value steps are serialised. The sustained throughput of such a design is given by Equation (3).

$$T_{seq} = f_{clk} / CPP \quad (3)$$

In Equation (3), T_{seq} is the sequential throughput in output pixels per second, f_{clk} is the clock frequency in hertz, and CPP is the number of clock cycles required per output pixel. At $f_{clk} = 100$ MHz and a representative $CPP = 8$, the sequential throughput is 1.25×10^7 pixels per second. This nominally exceeds the 640×480 target, but the margin shrinks quickly as resolution increases or as CPP grows with control overhead, and it leaves little headroom for higher frame rates.

2.3 Pipelined RTL Throughput

In the pipelined architecture every stage is registered and operates concurrently. Once the line buffers and pipeline registers have filled, the design accepts one input pixel and produces one output pixel on every clock cycle, so the steady-state throughput reduces to Equation (4).

$$T_{pipe} = f_{clk} \quad (4)$$

In Equation (4), T_{pipe} is the pipelined steady-state throughput in output pixels per second and f_{clk} is the clock frequency in hertz. At 100 MHz the design sustains 1.0×10^8 pixels per second, roughly ten times the requirement from Section 2.1 and an order of magnitude above the sequential estimate of Section 2.2.

Simulation confirmed this behaviour. In the 640×480 run the testbench streamed all 307,200 input pixels through the design with no stalls, one pixel per clock, and the full frame completed in 3.085 ms of simulated time. At 100 MHz this corresponds to approximately 324 frames per second, so the measured steady-state throughput matches Equation (4).

2.4 Latency Analysis

Latency is the delay between a pixel entering the accelerator and the corresponding output pixel becoming valid, and it is distinct from throughput. For this design the latency has three contributions: the line-buffer fill, the window-formation delay, and the arithmetic pipeline depth, as expressed in Equation (5).

$$L = 2W + D_w + D_a \quad (5)$$

In Equation (5), L is the total latency in clock cycles, W is the image width in pixels, D_w is the window-formation delay of the shift-register stage in cycles, and D_a is the depth of the arithmetic pipeline in cycles. Two full image rows ($2W = 1,280$ cycles at $W = 640$) must enter the line buffers before the first complete window column exists; window formation adds $D_w = 3$ cycles and the two registered arithmetic stages add $D_a = 2$ cycles, for a predicted latency of 1,285 cycles.

The measured latency in simulation was exactly 1,285 clock cycles (12.85 μ s at 100 MHz) from the first pixel clocked in to the first assertion of the output valid signal, which confirms the model in Equation (5). This latency is paid once at the start of the stream and does not reduce steady-state throughput; against a 33 ms frame period at 30 frames per second it is negligible.

2.5 Sobel Arithmetic Analysis

The Sobel operator estimates image gradients with two 3×3 kernels applied to the window pixels p_{00} through p_{22} , where p_{00} is the top-left pixel and p_{22} is the bottom-right pixel (Figure 2). The horizontal gradient is computed using Equation (6).

$$G_x = (p_{02} + 2p_{12} + p_{22}) - (p_{00} + 2p_{10} + p_{20}) \quad (6)$$

In Equation (6), G_x is the horizontal gradient estimate and the p_{ij} terms are the 8-bit pixel intensities at row i , column j of the window. The vertical gradient is computed using Equation (7).

$$G_y = (p_{20} + 2p_{21} + p_{22}) - (p_{00} + 2p_{01} + p_{02}) \quad (7)$$

In Equation (7), G_y is the vertical gradient estimate, with the p_{ij} terms defined as above. The kernel coefficients are limited to -2 , -1 , 0 , 1 , and 2 . Multiplication by two is a hard-wired left shift and multiplication by one requires no logic, so both gradients are implemented with adders, subtractors, and shifts only, avoiding DSP-block usage and reducing logic depth [4]. The edge magnitude is approximated by Equation (8).

$$M \approx |G_x| + |G_y| \quad (8)$$

In Equation (8), M is the approximate edge-magnitude output and $|G_x|$ and $|G_y|$ are the absolute values of the two gradients. The approximation avoids the square root of the true Euclidean magnitude at the cost of a small, well-understood directional error [4]. Bit-width growth is bounded rather than assumed: the largest possible gradient magnitude is $4 \times 255 = 1,020$, which

fits in a 13-bit signed word, and the sum $|G_x| + |G_y|$ is at most 2,040, held in 14 bits before the final saturation to the 8-bit output range. The implemented datapath uses exactly these widths, so no intermediate sum can overflow.

2.6 Resource Usage Analysis

FPGA resource usage will be finalised from Xilinx Vivado synthesis of the RTL [5]. The metrics of interest are look-up tables (LUTs), flip-flops, DSP blocks, and block RAM, together with the maximum clock frequency from static timing analysis. Table 1 lists the architectural estimates; each value follows directly from the structure of the design and will be replaced by the corresponding Vivado report figure once synthesis is run.

Table 1: Expected synthesis results (pre-synthesis architectural estimates, to be confirmed in Vivado).

Metric	Expected value	Basis of estimate
Look-up tables (LUTs)	$\approx 300\text{--}600$	adder/subtractor trees and control logic
Flip-flops (FFs)	$\approx 400\text{--}700$	window shift registers and pipeline registers
DSP blocks	0	no multipliers; coefficients use shifts and adds
Block RAM	$2 \times 18\text{ Kb}$	one $640 \times 8\text{-bit}$ row buffer per line buffer (5,120 bits each)
Maximum clock frequency	$> 100\text{ MHz}$	shallow logic between registers (2-stage arithmetic)

The two line buffers dominate memory use and are written in the read-first style that Vivado infers as block RAM [5]. Because the datapath contains no multipliers, DSP usage is expected to be zero by construction rather than by optimisation.

2.7 Verification Results

The RTL is verified by direct comparison against a software golden model that implements the same integer arithmetic in Python. A conversion script turns a source image into a text stream of pixel values, the VHDL testbench streams the pixels through the design and records the output stream, and the model then compares every output pixel [3]. Because the RTL and the golden model use identical integer arithmetic, any mismatch indicates a real design fault rather than rounding noise.

Four complete runs were performed at increasing frame sizes, including a synthetic directed-edge image and a full 640×480 frame. The results are summarised in Table 2. In all four runs every output pixel matched the golden model exactly: 100.0000 % exact match, zero maximum absolute error, and therefore infinite peak signal-to-noise ratio (PSNR).

Table 2: Verification results, RTL output versus software golden model (GHDL simulation).

Input frame	Output frame	Output pixels	Exact match	Max abs error	PSNR
-------------	--------------	---------------	-------------	---------------	------

32×32	30×30	900	100.0000 %	0	∞ dB
48×48	46×46	2,116	100.0000 %	0	∞ dB
256×256	254×254	64,516	100.0000 %	0	∞ dB
640×480	638×478	304,964	100.0000 %	0	∞ dB

Figure 4 shows the 640×480 case: the input image, the RTL output, the golden-model output, and the pixel-by-pixel difference image, which is uniformly zero. Reset and border behaviour are exercised by every run, since each simulation starts from reset and the valid logic must suppress the border pixels for which no complete 3×3 window exists; the output frame size of $(W - 2) \times (H - 2)$ pixels was produced exactly in all four runs.

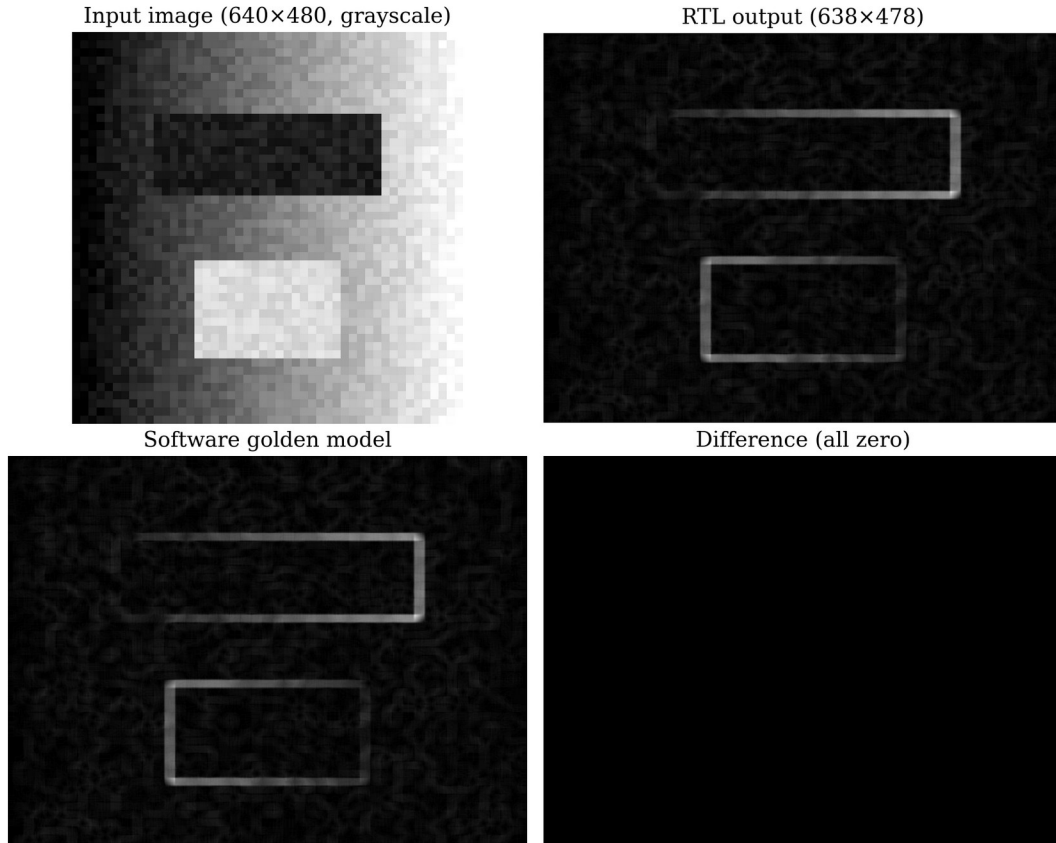


Figure 4: Verification of the 640×480 run: input, RTL output, golden model, and difference image.

2.8 Sensitivity and Limitations of the Analysis

The real-time margin is sensitive to three parameters. From Equations (2) and (4), the margin is the ratio of f_{clk} to $W \times H \times F$: raising the resolution or frame rate increases the requirement linearly, while a lower achievable clock frequency reduces the supply. The sequential alternative also scales inversely with CPP from Equation (3), which makes it far more sensitive to control overhead; the pipelined design keeps a large margin across plausible operating points.

The analysis itself has limits that should be kept in view. It covers grayscale images and the fixed 3×3 Sobel operator only; larger kernels would add line buffers and arithmetic depth, and RGB processing would widen or multiplex the datapath. The throughput and latency figures assume an ideal stall-free input stream, so a real camera or bus interface with back-pressure would need the ready/valid handshake noted in the proposed solution. The 100 MHz clock is a testbench assumption until Vivado timing analysis confirms it, and the resource figures in Table 1 remain estimates until synthesis. Finally, because the golden model shares its integer arithmetic with the RTL, the verification establishes that the hardware implements the intended computation; it does not evaluate how well the Sobel operator itself detects edges, which is a property of the algorithm rather than of this design.

3.0 Discussion

The measurements support the choice of the pipelined architecture. The design sustains one output pixel per clock cycle in simulation, which at 100 MHz is about ten times the 9.216×10^6 pixels-per-second requirement derived in Section 2.1 and an order of magnitude beyond the sequential estimate. Throughput is the quantity most directly tied to the engineering problem of deterministic real-time processing, so this margin is the decisive result.

The cost of that margin is modest. The extra hardware is dominated by the two line buffers, which fit in two 18 Kb block RAMs, and by pipeline registers; the arithmetic uses no multipliers at all. The design therefore trades a small, cheap amount of memory for a tenfold throughput margin, which is a favourable exchange on any modern FPGA.

The exact agreement between the predicted and measured latency (1,285 cycles in both cases) is worth emphasising, because it validates the analytical model of Equation (5) rather than just the simulation. Latency and throughput remain distinct quantities: the $12.85 \mu\text{s}$ fill delay is paid once per stream and is negligible against a 33 ms frame period, although a latency-critical closed-loop application would still need to budget for it explicitly.

Two results remain provisional. Timing closure has not yet been demonstrated: the throughput of Equation (4) holds only if the synthesised design meets the 100 MHz clock, and although the shallow two-stage arithmetic pipeline makes this likely, the claim is confirmed only by the Vivado timing report. Likewise the resource figures in Table 1 are architectural estimates until the synthesis utilisation report replaces them. Verification confidence is bounded by the test set: four image sizes provide strong evidence of correctness, but they do not constitute a formal proof.

Beyond the immediate project, the results generalise in useful ways. The measured 100 Mpixel/s capability already exceeds the 62.2×10^6 pixels-per-second demand of 1080p video at 30 frames per second, so the same architecture would support full-HD edge detection with no structural change beyond longer line buffers. The line-buffer-and-window pattern is not specific to Sobel: any 3×3 kernel, including Gaussian smoothing or sharpening, and the first convolutional layers

of neural networks reuse the same streaming structure, so both the design and the analysis method transfer directly to other streaming vision accelerators. The natural next steps are to run Vivado synthesis and timing analysis to replace the estimated figures, to capture simulation waveforms for documentation, and to add an AXI-Stream wrapper so the block can be dropped into a larger video pipeline.

References

- [1] D. G. Bailey, *Design for Embedded Image Processing on FPGAs*. Singapore: John Wiley & Sons (Asia), 2011.
- [2] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. New York, NY: Pearson, 2018.
- [3] T. Gingold, "GHDL Documentation," [Online]. [Cited 2026 July 3]. Available at: <https://ghdl.github.io/ghdl/>
- [4] N. Kanopoulos, N. Vasanthavada, and R. L. Baker, "Design of an image edge detection filter using the Sobel operator," *IEEE J. Solid-State Circuits*, Vol. 23, No. 2, pp. 358–367, Apr. 1988.
- [5] *Vivado Design Suite User Guide: Synthesis*, UG901 (v2024.1), Advanced Micro Devices, Inc., San Jose, CA, 2024.

Appendix A: Milestone Report I — Project Background

Milestone Report I is attached in full below, as required by the assignment guidelines, so that the project background, objective, potential solutions, and initial assessment are available to the reader. The project scope is unchanged since Report I.

ENGR 446 — Milestone Report I: Project Background

Design and Analysis of a Pipelined RTL Hardware Accelerator for Real-Time Sobel Edge Detection

Robin Zerr · V01013510 · Electrical Engineering

1.0 Background

Digital image processing relies heavily on two-dimensional (2D) convolution, an operation in which a small coefficient matrix, known as a kernel or mask, is slid across an image and combined with the underlying pixels through multiply-accumulate arithmetic. Each output pixel is the weighted sum of a local neighbourhood of input pixels, and the kernel coefficients determine the filtering behaviour, such as blurring, sharpening, or gradient estimation [1]. Because the same arithmetic pattern repeats for every pixel, 2D convolution is computationally regular but data-intensive, which makes it a natural candidate for hardware acceleration.

Sobel edge detection is a widely used application of 2D convolution. It estimates the horizontal and vertical intensity gradients of an image using two 3×3 kernels and combines them into an edge-magnitude image that highlights boundaries between regions. Sobel filtering is favoured in embedded vision because it is computationally inexpensive relative to its usefulness, provides reasonable noise immunity through its built-in smoothing term, and maps efficiently onto fixed-function arithmetic [2].

Real-time image processing imposes a hard constraint: every pixel of every frame must be processed within a fixed time budget. A general-purpose central processing unit (CPU) executes the convolution as a sequence of instructions, and its throughput is limited by instruction fetch, memory access, and cache behaviour, all of which introduce non-deterministic latency. For embedded vision systems operating at sustained frame rates, this variability can prevent the system from meeting its timing budget.

Field-programmable gate arrays (FPGAs) and register-transfer-level (RTL) architectures address this limitation by implementing the convolution directly as dedicated digital logic. Rather than executing instructions, an RTL accelerator forms a spatial datapath that processes pixels as they stream through, exploiting both pipelining and parallelism, and allowing the designer to trade silicon area for throughput in a controlled and deterministic way [3].

Four engineering quantities dominate the design of such an accelerator. Throughput measures how many output pixels are produced per unit time and determines whether real-time frame rates

can be sustained. Latency measures the delay between a pixel entering and the corresponding result leaving the pipeline, and matters for closed-loop or interactive systems. Resource usage measures the look-up tables (LUTs), flip-flops (FFs), digital signal processing (DSP) blocks, and block RAM (BRAM) consumed, which constrains cost and scalability. Timing closure refers to whether the synthesised circuit can meet its target clock period; a design that cannot close timing cannot run at the intended frequency regardless of its architectural merit.

2.0 Objective

2.1 Problem

Real-time Sobel edge detection requires deterministic, sustained processing of a continuous pixel stream. A processor-based implementation may not provide the guaranteed throughput and bounded latency needed for embedded vision at standard video resolutions and frame rates, because instruction execution and memory access introduce variability. A dedicated RTL hardware accelerator is therefore required, but it is not obvious which RTL architecture best satisfies the throughput, latency, resource, and timing requirements of the application.

2.2 Purpose

The purpose of the final report is to determine the most suitable RTL hardware architecture for a real-time 3×3 Sobel edge-detection accelerator that processes a streaming grayscale image source.

2.3 Aims

The aims of the project are to:

1. Support at least 640×480 grayscale video at 30 frames per second, equivalent to a sustained pixel rate of 9.216 million pixels per second.
2. Compare a sequential and a pipelined RTL architecture using cycles per output pixel as the primary throughput metric.
3. Estimate or measure FPGA resource usage (LUTs, flip-flops, DSP blocks, and BRAM) using Xilinx Vivado synthesis [4].
4. Determine whether the selected design can meet a clock frequency sufficient for real-time processing of the target video stream.
5. Verify the RTL output against a software reference (golden) model to a defined accuracy.

2.4 Limitations

The scope of the project is bounded as follows:

- Only grayscale (single-channel) images are processed; RGB colour processing is excluded.

- Only the standard 3×3 Sobel operator is considered; other kernels and kernel sizes are excluded.
- No physical camera interface is required; inputs are stored images or test vectors.
- No full application-specific integrated circuit (ASIC) layout is performed.
- No neural-network inference or learned filtering is included.
- The implementation may be simulated using stored image and test-vector inputs rather than live video.

3.0 Potential Solutions

3.1 Sequential RTL Filter

The sequential RTL filter loads or assembles a 3×3 pixel window and then performs the Sobel arithmetic over several clock cycles using a small number of shared arithmetic resources. A finite-state machine sequences the multiply-accumulate steps for the horizontal and vertical gradients, accumulating partial sums in registers before producing one output pixel. Because relatively few operations occur in parallel, the design uses modest logic resources and is straightforward to design, debug, and verify. Its principal limitation is throughput: each output pixel requires multiple clock cycles, so the sustained pixel rate is the clock frequency divided by the cycles per pixel, which may be insufficient for real-time operation at the target resolution and frame rate.

3.2 Pipelined RTL Filter

The pipelined RTL filter uses line buffers and shift registers to continuously assemble a 3×3 pixel window directly from the incoming pixel stream. Two line buffers store the two most recent image rows so that the three vertically aligned pixels of each window column are available simultaneously; three shift registers then provide the three horizontal taps [3]. The Sobel arithmetic is partitioned into pipeline stages so that, after an initial line-buffer and pipeline-fill delay, the circuit produces one filtered output pixel per clock cycle. This architecture is more complex and consumes more registers and memory, but it directly targets the high-throughput, low-latency, streaming nature of the problem.

4.0 Initial Assessment

The two candidate architectures are compared using a weighted decision matrix. Seven criteria were selected because they map directly onto the project aims and onto the engineering quantities introduced in Section 1.0. Each criterion is assigned a weight reflecting its importance to the objective, and each candidate is scored from 1 (poor) to 5 (excellent). The weighted score for each candidate is the sum of the products of weight and score, as defined in Equation (1).

$$S = \sum (w_i \times r_i) \quad (1)$$

In Equation (1), S is the total weighted score for a candidate architecture, w_i is the weight assigned to criterion i , and r_i is the score awarded to that candidate for criterion i . The summation is taken over all seven criteria. The scores are preliminary engineering estimates used to guide solution selection and will be revisited once Vivado synthesis results are available.

The weighting prioritises throughput and latency most heavily because the central engineering problem is sustaining a deterministic real-time pixel rate. Timing feasibility is also weighted highly, because a design that cannot close timing cannot meet the rate regardless of its theoretical throughput. Hardware resource usage carries moderate weight, since the target device has finite logic and memory. Implementation and verification complexity are weighted lower because they affect development effort rather than whether the delivered accelerator meets its requirements. Relevance to digital-design industry skills is included as a modest educational criterion.

Table 1: Weighted decision matrix for selecting the RTL Sobel architecture.

Criterion	Weight	Seq. r	Seq. $w \times r$	Pipe. r	Pipe. $w \times r$
Throughput	0.25	2	0.50	5	1.25
Latency	0.15	3	0.45	4	0.60
Hardware resource usage	0.15	5	0.75	3	0.45
Timing feasibility	0.15	4	0.60	4	0.60
Implementation complexity	0.10	5	0.50	2	0.20
Verification complexity	0.10	4	0.40	3	0.30
Relevance to industry skills	0.10	3	0.30	5	0.50
Total weighted score (S)	1.00		3.50		3.90

As summarised in Table 1, the pipelined RTL filter achieves the higher total weighted score, driven mainly by its superior throughput, which is the most heavily weighted criterion. The sequential design scores well on resource usage and simplicity but is penalised heavily on throughput, the criterion most directly tied to the engineering problem. On the basis of this initial assessment, the pipelined RTL filter is selected as the proposed solution, subject to confirmation by Vivado synthesis and timing results in the later milestones.

References

- [1] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. New York, NY: Pearson, 2018.
- [2] N. Kanopoulos, N. Vasanthavada, and R. L. Baker, "Design of an image edge detection filter using the Sobel operator," *IEEE J. Solid-State Circuits*, Vol. 23, No. 2, pp. 358–367, Apr. 1988.
- [3] D. G. Bailey, *Design for Embedded Image Processing on FPGAs*. Singapore: John Wiley & Sons (Asia), 2011.

[4] *Vivado Design Suite User Guide: Synthesis*, UG901 (v2024.1), Advanced Micro Devices, Inc., San Jose, CA, 2024.